I. Dokumentacja do lab. nr 3 - "Modułowość. Pakiety. Mechanizm importowania klas."

II. Roch Majkowski - grupa 1, semestr III

III. Przedmiot - "Programowanie obiektowe"

IV. Opis zadania do realizacji

Do zrealizowania były następujące zadania:

1. Utwórz strukturę pakietów pl.edu.lab.model, pl.edu.lab.service, pl.edu.lab.app i przenieś klasy do odpowiednich pakietów.
2. Zaimplementuj klasę Product w pakiecie model oraz klasę ProductService w pakiecie service, która operuje na liście produktów.
3. W klasie Main (pakiet app) użyj importów jawnych i uruchom scenariusz dodawania oraz wypisywania produktów.
4. Pokaż różnicę między import java.util.List; a import java.util.*; i opisz, które podejście preferujesz w projekcie i dlaczego.
5. Rozdziel projekt na co najmniej 5 klas tak, aby każda miała jedną odpowiedzialność i była w logicznym pakiecie.
6. Dodaj krótką instrukcję uruchomienia projektu z pakietami (kompilacja i uruchomienie klasy głównej) w formie 4-6 punktów.

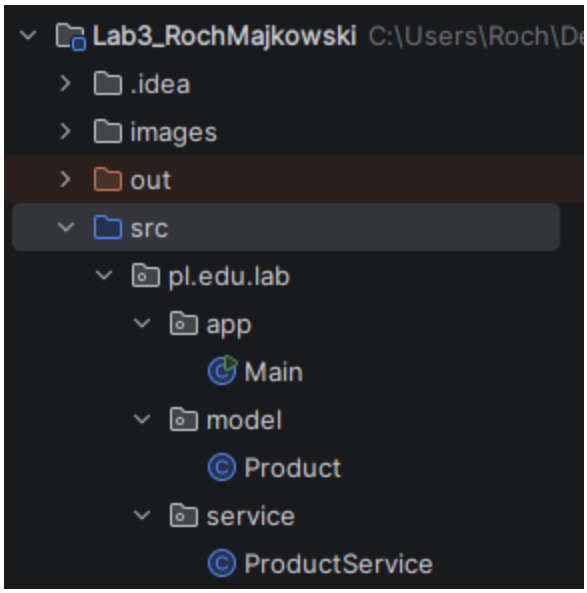
V. Technologie wykorzystane w zadaniu

- Java

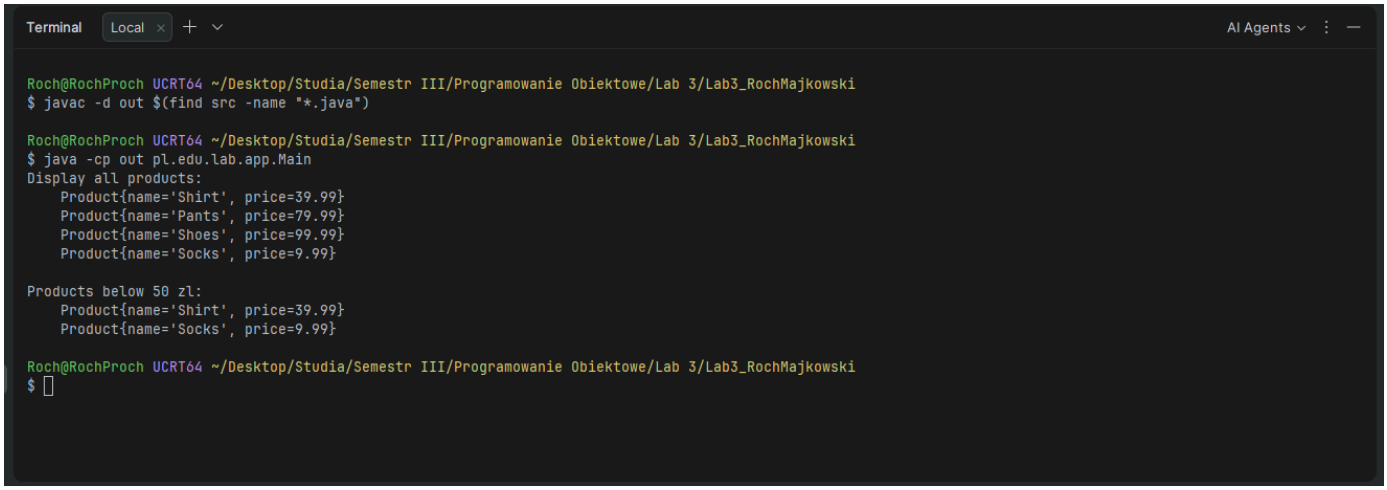
VI. Realizacja zadania

1. Zadanie - Utworzenie odpowiedniej struktury pakietów i deklaracja zgodna z katalogiem.

1a. Odpowiadająca nazwa pakietów:



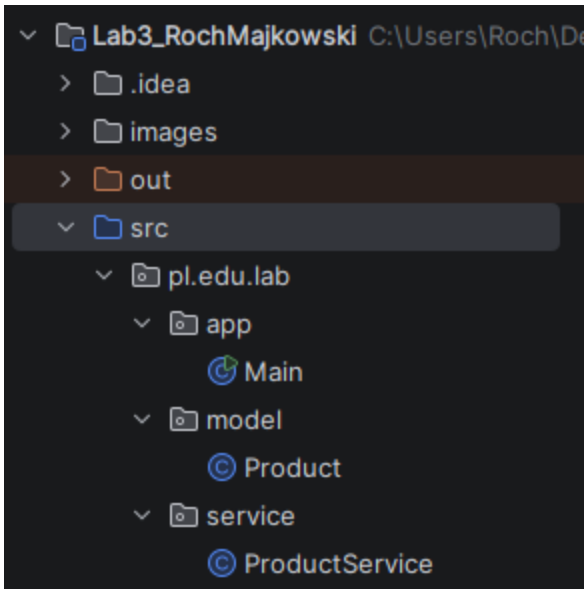
1c. Poprawne kompilowanie programu:



2. Zadanie - Klasy Product i ProductService.

- Klasa Product jest w pakiecie pl.edu.lab.model
- klasa ProductService jest w pakiecie pl.edu.lab.service
- Obie importują 'package' zgodny z katalogiem.

2a. Lokalizacja plików:



Kod wykorzystany do rozwiązania zadania (zadań):

Klasa Product

```
package pl.edu.lab.model;

public class Product {
    private String name;
    private Double price;

    public Product(String name) {
        this.name = name;
    }

    public Product(String name, Double price) {
        this.name = name;
        this.price = price;
    }

    public String getName() { return name; }
    public Double getPrice(){ return price; }

    @Override
    public String toString() {
        return "Product{" +
                "name='" + name + '\'' +
                ", price=" + price +
                '}';
    }
}
```

Klasa ProductService

```
package pl.edu.lab.service;
```

```java
import pl.edu.lab.model.Product;

import java.util.*;
//import java.util.ArrayList;
//import java.util.List;

public class ProductService {

    private List<Product> products = new ArrayList<>();

    public void addProduct(Product pd) {
        products.add(pd);
    }

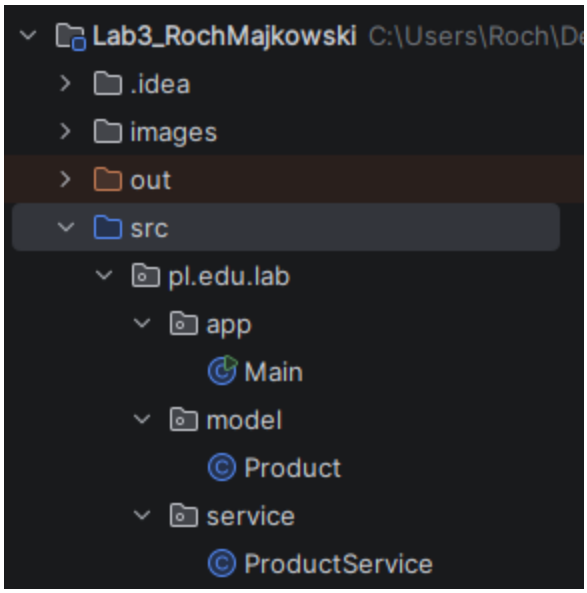
    public List<Product> filterByPrice(double maxPrice) {
        List<Product> output = new ArrayList<>();
            for (Product pd : products){
                if(pd.getPrice() <= maxPrice) {
                    output.add(pd);
                }
            }
            return output;
    }

    public void displayAll() {
        for (Product pd : products) {
            System.out.println("    " + pd);
        }
    }
}
```

3. Zadanie - Klasa Main i scenariusz użycia

- Klasa Main jest w pakiecie pl.edu.lab.app, zawierając metodę main(String[] args)
- Używa importów jawnych
- Tworzy kilka obiektów Product, dodając je do ProductService - wypisując wynik w konsoli.

3a. Lokalizacja plików:



Kod wykorzystany do rozwiązania zadania (zadań):

```
package pl.edu.lab.app;

import pl.edu.lab.model.Product;
import pl.edu.lab.service.ProductService;

public class Main {
    public static void main(String[] args) {

        ProductService service = new ProductService();
        service.addProduct(new Product("Shirt", 39.99));
        service.addProduct(new Product("Pants", 79.99));
        service.addProduct(new Product("Shoes", 99.99));
        service.addProduct(new Product("Socks", 9.99));

        System.out.println("Display all products: ");
        service.displayAll();

        System.out.println("\nProducts below 50 zl: ");
        for (Product pd : service.filterByPrice(50)) {
            System.out.println("    " + pd);
        }
    }
}
```

3b. Poprawne kompilowanie programu:

```
Terminal  Local                                                    AI Agents

Roch@RochProch UCRT64 ~/Desktop/Studia/Semestr III/Programowanie Obiektowe/Lab 3/Lab3_RochMajkowski
$ javac -d out $(find src -name "*.java")

Roch@RochProch UCRT64 ~/Desktop/Studia/Semestr III/Programowanie Obiektowe/Lab 3/Lab3_RochMajkowski
$ java -cp out pl.edu.lab.app.Main
Display all products:
    Product{name='Shirt', price=39.99}
    Product{name='Pants', price=79.99}
    Product{name='Shoes', price=99.99}
    Product{name='Socks', price=9.99}

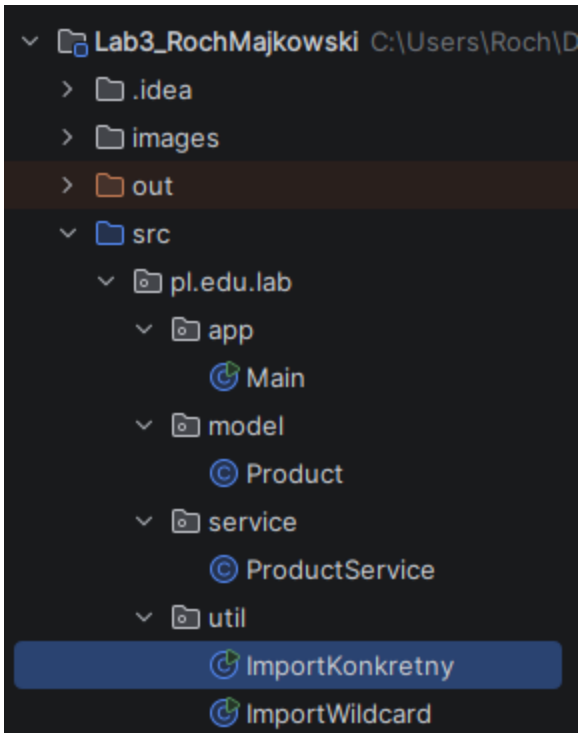
Products below 50 zl:
    Product{name='Shirt', price=39.99}
    Product{name='Socks', price=9.99}

Roch@RochProch UCRT64 ~/Desktop/Studia/Semestr III/Programowanie Obiektowe/Lab 3/Lab3_RochMajkowski
$
```

4. Zadanie - Import konkretny a Wildcard

- Wszystkie zaimportowane klasy korzystając z importów konkretnych są konkretne dla naszych przeznaczonych działań, lecz czasami wypada wymóg ręcznego dodania ich - chociaż wiemy przynajmniej z jakimi klasami pracujemy i co one robią.
- Wildcard importuje np. wszystkie klasy dostępne w klasie java.util, czyli .List, .ArrayList, .Map, .HashMap zostały zaimportowany, ALE zostały też zaimportowane zbędne klasy z java.util
- Wildcard jest przydatny przez wygodę użytku (czasami edytor sam nie dodaje klas i musimy je ręcznie dodać podczas kodowania), natomiast dodaje zbędne (w tym przypadku) klasy, których nie potrzebujemy do programu
- Wildcard jest o wiele wygodniejszy od konkretnego importowania, nawet jak nie korzystamy z niepotrzebnych narzędzi. Wole ten wybór, nawet jak program może dłużej się kompilować.

4a. Lokalizacja plików:



Kod wykorzystany do rozwiązania zadania (zadań):

Klasa ImportKonkretny

```
package pl.edu.lab.util;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

// Wszystkie zaimportowane klasy są konkretne dla naszych przeznaczonych działań, lecz czasami wypada wymóg ręczne

public class ImportKonkretny {

    public static void main(String[] args) {
        List<String> lista = new ArrayList<>(); // Zaimportowany statycznie java.util.List i java.util.ArrayList

        lista.add("Jan");
        lista.add("Ala");
        lista.add("Paweł");

        Map<String, Integer> mapa = new HashMap<>(); // Zaimportowany statycznie java.util.Map i java.util.HashMap
        mapa.put("Jan", 34);
        mapa.put("Ala", 42);
        mapa.put("Paweł", 16);

        System.out.println("Lista osób: " + lista);
        System.out.println("Informacje o osobach: " + mapa);

    }

}
```

Klasa ImportWildcard

```
package pl.edu.lab.util;

import java.util.*; // Importuje wszystkie klasy dostępne w klasie java.util, czyli .List, .ArrayList, .Map, .Hash
// Wildcard jest przydatny przez wygodę użytku (czasami edytor sam nie dodaje klas i musimy je ręcznie dodać podcz
// Wildcard jest o wiele wygodniejszy od konkretnego importowania, nawet jak nie korzystamy z niepotrzebnych narzę
public class ImportWildcard {

    public static void main(String[] args) {
        List<String> lista = new ArrayList<>();

        lista.add("Adam");
        lista.add("Grzegorz");
        lista.add("Bartek");

        Map<String, Integer> mapa = new HashMap<>();
        mapa.put("Adam", 27);
        mapa.put("Grzegorz", 53);
        mapa.put("Bartek", 12);
```

```
        System.out.println("Lista osób: " + lista);
        System.out.println("Informacje o osobach: " + mapa);

    }

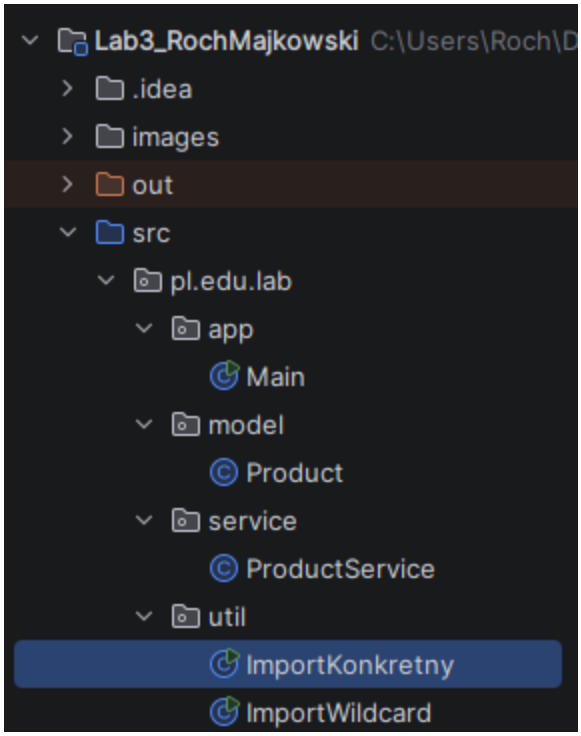
}
```

5. Zadanie - Podział projektu na 5 odrębnych klas

Klasy w projekcie zostały podzielone między folderami w ścieżce `pl\edu\lab` i każda z nich ma określoną odpowiedzialność:

- Klasa Main kompiluje program i wypisuje dane wzięte z pozostałych klas (Product, ProductService)
- Klasa Product posiada konstruktory i gettery, które pozwalają na utworzenie obiektów
- Klasa ProductService posługuje się klasą Product, żeby przechowywać listę obiektów Product, udostępniając metody do dodawania oraz wypisowania/filtrowania produktów.
- Klasa ImportKonkretny bierze przykładowy kod i pokazuję różnicę w strukturze, w jaki sposób importuje się klasy z danego pakietu
- Klasa ImportWildcard, robi podobną rzecz co Klasa ImportKonkretny, ale wskazuje na przykład zastosowania Wildcard'a (*) zamiast importów konkretnych.

5a. Lokalizacja plików:



6. Instrukcja uruchomienia projektu

Poniżej znajduje się instrukcja uruchomienia programu:

1. Uruchomić program 'IntelliJ IDEA' i ustawienie terminalu na GitBash, poprzez wejście w Ustawienia (`CTRL+ALT+S`).

2. Znalezienie w ustawieniach `Tools -> Terminal` i zmiana Shell path'a na GitBash i zastosowanie zmian, żeby polecenia poprawnie działały.
3. Otworzenie Terminalu poprzez wejście w `View -> Tool Windows -> Terminal` , lub użycia skrótu klawiszowego `ALT+F12` .
4. Otwórz terminal w głównym katalogu projektu (tam, gdzie znajduje się folder `src`).
5. Wpisanie polecenia `javac -d out $(find src -name "*.java")` , gdzie flaga `-d` wskazuje katalog docelowy dla plików.
6. Uruchomienie programu poprzez wpisanie polecenia `java -cp out pl.edu.lab.app.Main` i sprawdzenie czy kompiluje się program. Opcja `-cp out` mówi gdzie szukać skompilowanych klas, gdzie trzeba podać nazwę pakietu i nazwę klasy, ponieważ szuka klasy w katalogu z classpath (`out/pl/edu/lab/app/Main.class`).

6a. Poprawne kompilowanie programu i zastosowanie poleceń:

